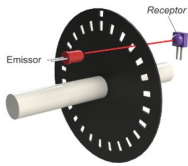


Universidade Federal de Itajubá
Instituto de Engenharia de Sistemas e Tecnologias da Informação - IESTI
2a Prova de ECOP04 – Programação para Sistemas Embarcados 2º sem. 2024
Prof. Rodrigo Maximiano Antunes de Almeida, Prof. Décio Rennó de Mendonça Faria



(25 pts) Questão 1: Em uma indústria, existe um sistema que permite controlar a quantidade de óleo utilizado em um processo. Esse sistema funciona com um motor (ligado à porta B, bit 2) que gira um dosador. A cada grau que o sistema gira, uma certa quantidade de óleo é inserida no sistema. Para ter um controle mais preciso da quantidade, usamos um disco perfurado com um sensor ótico. Cada vez que o sinal passa de zero para 1 nesse sensor (ligado à porta B bit 0) dizemos que o sistema girou 1 pulso. Um segundo sensor (ligado à porta B bit 1) verifica se a quantidade de óleo na máquina é suficiente (ligado) ou não (desligado). Caso ele detecte que está faltando óleo o sistema deve acionar um alarme (ligado à porta A bit 0). Caso contrário, o alarme deve ser desligado.

Monte apenas a **biblioteca** de interface para esse sistema com 3 funções:

```
void VazaoInit(void); // Inicializa o sistema configurando as entradas e saídas
void VazaoInjeta(unsigned int pulsos); // Liga o motor e espera uma quantidade de pulsos e desliga o motor
void VazaoStatus(void); // Verifica se o sensor de óleo está ativado e liga/desliga o alarme
```

(25 pts) Questão 2: Um sistema de iluminação utiliza um sinal de PWM para controlar o brilho dos LEDs. Para isso o usuário pode ajustar a luminosidade através de um potenciômetro ligado ao ADC do microcontrolador. O sistema deve ler o valor no conversor AD e configurar o sinal PWM para o brilho escolhido. Quando o potenciômetro está no máximo, o conversor AD fornece o seu valor máximo (1023) e quando está no mínimo, o valor 0 (zero). Quando o sinal PWM é igual a 0 (zero) os LEDs estão apagados e quanto o sinal PWM é 100%, o brilho é máximo. Para ter uma referência durante o ajuste, uma chave (porta A bit 3) desativa o controle do PWM pelo potenciômetro e fixa os LEDs no valor de 70% de brilho (valor fixo). Implemente um **programa** que faça a leitura do potenciômetro e da chave e controle o brilho dos LEDs via PWM.

(25 pts) Questão 3: Um painel de LCD é utilizado para mostrar a quantidade de veículos que passam em uma praça de pedágio. Este painel recebe pela porta serial apenas dois comandos: 'S' (somar 1) e 'Z' (zerar a contagem). O valor atual da contagem deve ser apresentado com 6 dígitos na primeira linha do display de LCD. Além dos comandos recebidos pela porta serial, um botão no painel do equipamento (porta B bit 0) também permite zerar a contagem. Faça um programa que utiliza a serial, o LCD e o botão para implementar esse dispositivo.

(30 pts) Questão 4: Um laboratório de química utiliza um cronômetro que mostra o tempo de reação de um experimento em um display de 7 seguimentos com quatro dígitos. Dois dígitos são utilizados para segundos e os dois restantes para minutos. O cronômetro possui um botão que permite zerar o tempo (porta B bit 0) e um botão que permite iniciar e parar a contagem (porta B bit 1). Implemente o **programa** utilizando a interrupção do Timer para controlar o tempo do relógio.

```
char flag;
//Esta interrupção é chamada a cada 1ms.
void Int(void) __interrupt(){
    if (BitTst(INTCON, 2)) { //Verifica se ocorreu overflow de TMR0
        timerReset(1000); //Ajusta o contador do TMR0
        BitClr(INTCON, 2); //Desliga o Flag de overflow do TMR0
        flag = 1;
    }
}
```

Obs: A função InitInterrupt() ajusta todos os parâmetros da interrupção.

```
//adc.h
void adcInit(void);
int adcRead(void);

//lcd.h
void lcdInit(void);
void lcdChar(char letra);
void lcdCommand(char val);

//pwm.h
void pwmInit(void);
void pwmFrequency(unsigned int freq);
void pwmSet1(unsigned char porcento);

//config.h
Necessário para configurar o hardware
Possui comandos: #pragma ...
```

```
//pic18f4520.h
Define TRISA, PORTA,... bem como:
BitSet(arg,bit) BitClr(arg,bit)
BitFlp(arg,bit) BitTst(arg,bit)

//serial.h
void serialInit(void);
unsigned char serialRead(void);
void serialSend(unsigned char c);

//ssd.h
void ssdInit(void);
void ssdUpdate(void);
void ssdDigit(int val, int pos);

//Observação: TRIS = 0 → Saída
//Em ASCII caractere '0' = 48
```